DigitalDetective

CLI tool (cli.py)

User can search by full name, ip address and username.

Tool saves the result and shows visual output.

It has three options:

1 Name

def perform_name_search

Scrape (name_seach.py)

Validate and analyze the results (cli_name.py)

def get_personal_info

(https://serpapi.com)

def get_personal_info_from_fonecta

(https://www.fonecta.fi with BeautifulSoup)

def cross_reference_name_results

Visual table

def_display_name_results

helper functions:

def extract_json_from_html

def extract_fonecta_data

def extract_phone_numbers

2 IP

def perform_ip_search

Scrape (ip_search.py)

Validate and analyze the results (cli_ip.py)

def get_ip_info_ipapi

(http://ip-api.com)

def get_ip_info_iplocation

(https://api.iplocation.net)

def cross_reference_ip_results

Visual table

def_display_ip_results

3 Username

def perform_username_search

Scrape (username_search.py)

Validate and analyze the results (cli_username.py)

def get_username_mentions_analyzer

((https://github.com/qeeqbox/social-analyzer)

def get_username_mentions_sherlock

(https://github.com/sherlock-project/sherlock.git)

def cross_reference_username_results

Visual table

def_display_username_results

CLI tool commands

python3 src/cli.py --h

```
usage: cli.py [-h] [-n NAME] [-ip IP] [-un USERNAME]

Digital Detective CLI Tool
options:
 -h, --help                    show this help message and exit

 -n NAME,                      Performs a full-name search,
--name NAME                                   "first last"

 -ip IP, --ip IP               Performs an IP search, 1.2.3.4

 -un USERNAME,                 Performs a username search,
--username USERNAME                           "@username"
```

CLI tool commands via Virtual Environment

Commands and example results (commands.md)

source venv/bin/activate

Run different commands

```
((venv) student@student-ThinkPad-T490s:~
/digital-detective$ python3 src/cli.py -ip 8.8.8.8
Using cached data for ipapi_8.8.8.8
Using cached data for iplocation_8.8.8.8
                    IP Search Results
```

Field	Value
IP Address	8.8.8.8
ISP	Google LLC
Location	39.03, -77.5
Cross-Reference	Accurate: Both APIs returned matching or complementary results.

```
Result written to file: results/ip_8_8_8_8.txt
(venv)
```

deactivate

run_tests.sh via Virtual Environment

```
chmod +x run_tests.sh

./run_tests.sh
```

Automated tests (run_tests.sh)

Features

(What it does and how it works?)

Rate Limiting

- Implements delays and caching to handle API rate limits.
- Introduces delays using time.sleep in rate_cache_time.py.
- Caches API responses in the api_cache directory to reduce redundant requests.

Secure API Key Management

- Protects sensitive information using environment variables.
- Loads API keys from the .env file using the `dotenv` library in utils.py.

Error Handling

- Manages API errors and cases where no data is found.
- Implements `try-except` blocks in all API-related functions.
- Provides user-friendly error messages in the terminal.

Cross-Referencing

- Validates data accuracy by comparing results from multiple APIs.
- Compares fields like name, address, and phone number for name searches.
- Matches ISP and location data for IP searches.
- Checks for consistency in username mentions across platforms.

Tools and libraries

(Purpose, usage and why chosen?)

Sherlock

- Searches for usernames across platforms
- Runs as a subprocess in username_search.py to check for username mentions.
- Results are saved in the sherlock_results directory.
- Sherlock is a widely used OSINT tool for username enumeration.

Social Analyzer

- Searches for username mentions across multiple platformsU
- Used in `username_search.py` to complement Sherlock by providing additional insights into username mentions.
- Supports a wide range of platforms and provides detailed analysis

SerpAPI

- Fetches Google search results
- Used in `name_search.py`
- provides structured and reliable Google search results, simplifying data extraction

api_cache

- Caches API responses to reduce redundant requests and handle rate-limiting.
- Implemented in rate_cache_time.py to store and retrieve cached responses.
- Improves performance and ensures compliance with API rate limits.

time

- Implements delays to handle rate-limiting.
- Used in rate_cache_time.py to introduce delays between API requests.
- Simple and effective for managing time-based operations.

json

- Parses and processes JSON responses from APIs.
- Built-in Python library for handling structured JSON data.

dotenv

- Manages sensitive information (e.g., API keys) securely using environment variables.
- Loads API keys from the .env file in utils.py.
- Ensures sensitive information is not hard-coded into the source code.

requests

- Handles HTTP requests to APIs.
- Used in all API-related modules (e.g., name_search.py, ip_search.py) to fetch data.
- Simple and reliable for making HTTP requests.

argparse

- Parses command-line arguments for the CLI tool.
- Used in cli.py to handle user input and call the appropriate search functions.
- Built-in Python library for command-line argument parsing.

BeautifulSoup

- Scrapes personal information (e.g., name, address, phone) from public directory websites.
- Used in name_search.py to parse HTML responses and extract relevant data.
- BeautifulSoup is lightweight and efficient for web scraping tasks.

rich

- Creates visually appealing ASCII tables and styled terminal output.
- Used in CLI modules (e.g., cli_ip.py, cli_name.py, cli_username.py) to display search results in a structured and colorful format.
- Enhances the user experience by making terminal output more readable and engaging.

re

- Validates and extracts phone numbers using regex in name search.

Ethical consedarations

(Conserns and solutions)

Web Scraping

- Scraping websites without permission can violate terms of service and overload servers.
- The tool only scrapes publicly available data.
- It respects robots.txt files and avoids excessive requests to servers.

Data Privacy

- Collecting personal information can infringe on privacy rights.
- The tool only gathers data that is publicly available and voluntarily shared by users.
- It does not store or share data beyond the scope of the project.

Sensitive Information Protection

- Hard-coding API keys in the source code can expose them to unauthorized access.
- API keys are stored in a .env file and accessed securely using the dotenv library.
- The .env file is excluded from version control using .gitignore.

Ethical Use of Data

- OSINT tools can be misused for malicious purposes.
- The tool is intended for educational and ethical purposes only.
- Users are encouraged to comply with local laws and ethical guidelines when using the tool.

Rate Limiting

- Exceeding API rate limits can lead to throttling or bans.
- The tool implements delays between API requests using time.sleep.
- It caches API responses to minimize redundant requests.

Name search

- Scraping personal information from websites may violate privacy laws, terms of service.
- Tool uses BeautifulSoup to extract publicly available data but respects anti-scraping measures such as CAPTCHAs and dynamic content loading.
- The tool is intended for ethical use only and should not be used to find sensitive information like private addresses or phone numbers.

OSINT Open Source Intelligence

- Gathering publicly available information for actionable intelligence. Identifying vulnerabilities or leaked information.
- Includes websites, social media, public records, forums, and APIs.
- The information is publicly available and does not require hacking or unauthorized access.
- in the Project it relies on APIs and web scraping to gather publicly available data.

CIA confidentiality, integrity, availability

- Ensures that sensitive information is accessible only to authorized individuals. API keys are stored securely in a .env file and excluded from version control using .gitignore.
- Ensures that data is accurate and has not been tampered with. Cross-referencing results from multiple APIs validates the accuracy of the gathered data.
- Ensures that systems and data are accessible when needed. Cached API responses improve availability by reducing dependency on live API calls.

Digital footprint

- The trail of data left by individuals online, which can reveal sensitive information.
- Active Footprint: Data intentionally shared by users, such as social media posts, comments, and uploaded photos.
- Passive Footprint: Data collected without the user's explicit knowledge, such as IP addresses, browsing history, and cookies.
- It can be used for both legitimate purposes (e.g., targeted advertising) and malicious purposes (e.g., identity theft).

Cached data

FETCHING DATA:

```
(venv) student@student-ThinkPad-T490s:~/digital-detective$ python3 src/cli.py -ip 8.8.8.8
Request URL: http://ip-api.com/json/8.8.8.8
Request completed in 0.55 seconds
Request URL: https://api.iplocation.net/?ip=8.8.8.8&format=json
Request completed in 2.27 seconds
                    IP Search Results
┌──────────────────────────────┬──────────────────────────────
```

Name search functionality

1. Using SerpAPI for Google Search

- **Ethical**:** The tool uses SerpAPI, which is an official API designed for automated use. As long as the tool complies with SerpAPI's terms of service and only retrieves publicly available data, this part of the tool is ethical.
- **Legal**:** Since SerpAPI provides structured data in a compliant manner, using it is legal. The tool responsibly handles the absence of data by printing "Not found," which avoids misrepresentation or misuse of information.

2. Cross-Referencing with Fonecta Using BeautifulSoup

- **Ethical**:**
  - The tool uses BeautifulSoup to extract data from Fonecta, but it must respect Fonecta's **Terms of Service (ToS)**.
  - The tool does not bypass anti-scraping measures like CAPTCHAs or JavaScript-based content loading, which ensures compliance with ethical guidelines.
  - It only retrieves **openly searchable and publicly available data**, which is ethical as long as it does not involve sensitive or private information.
- **Legal**:**
  - Scraping data from Fonecta is legal only if it complies with Fonecta's ToS and does not violate privacy laws.
  - The tool avoids storing or sharing sensitive data beyond its intended use, which aligns with privacy regulations like GDPR or CCPA.

3. Responsible Data Handling

The tool demonstrates responsible handling of data by:
1. **Printing "Not found"** when no data is available, avoiding false positives or misleading results.
2. **Censoring sensitive data** (e.g., masking phone numbers) if necessary, ensuring that private information is not exposed.
3. **Cross-referencing results** from multiple sources (Google and Fonecta) to validate accuracy, which reduces the risk of errors or misuse.

4. Compliance with Ethical and Legal Guidelines

The tool adheres to the following principles:

1. **Respect for Terms of Service:**
  - The tool does not bypass anti-scraping measures like CAPTCHAs or JavaScript rendering.
  - It respects the ToS of websites like Fonecta and SerpAPI.

2. **Transparency:**
  - The tool's documentation (e.g., README) includes clear disclaimers and guidelines for ethical use.
  - Users are informed about their responsibilities to comply with applicable laws and website ToS.

3. **Privacy Protection:**
  - The tool only collects and displays publicly available information.
  - It avoids storing or sharing sensitive personal data (e.g., private phone numbers or addresses) without explicit consent.

Conclusion

The **name search functionality is ethical and legal **if****

1. It complies with the terms of service of SerpAPI and Fonecta.
2. It does not bypass anti-scraping measures or access restricted data.
3. It only retrieves and displays publicly available information.
4. It handles data responsibly, printing "Not found" when no data is available and censoring sensitive information when necessary.

IP search

- **Public IPs**

8.8.8.8 (Google)
1.1.1.1 (Cloudflare)
185.199.108.153 (Fastly)

The script successfully fetches data from both APIs and cross-references the results.

- **Private IPs**

192.168.1.1 (Local network)
10.0.0.1 (Private network)

These IPs are private and not routable on the public internet. The APIs return no meaningful data.

- **Loopback IP**

127.0.0.1

This returns no meaningful data because it is not routable on the public internet or associated with any ISP or geographic location. It is used to test network software or configurations on the local machine without requiring a physical network connection.

- **Edge Cases**

999.999.999.999 (Invalid format)
abcd (Non-IP string)

The APIs return no data because the IPs are invalid.

(Empty input)

Running the script without providing an IP (`python3 cli.py -ip`).

001.001.001.001 (IPs with leading zeros)

The APIs handle the IP correctly, but no meaningful data is returned.

Key Points About ISPs

1. Role of an ISP
- ISPs provide the infrastructure and services needed to connect to the internet.
- They assign IP addresses to devices on their network.

2. Examples of ISPs
- **Global ISPs**: Comcast, AT&T, Verizon, Vodafone, etc.
- **Regional ISPs**: Smaller companies that operate in specific areas.

3. Services Offered by ISPs
- Internet access (broadband, fiber, DSL, satellite, etc.).
- Email hosting.
- Web hosting and domain registration.
- Cloud services and VPNs.

4. How ISPs Are Identified
- When you query an IP address using APIs like `ip-api` or `iplocation.net`, they return the ISP associated with that IP.
- For example:
  - IP `8.8.8.8` → ISP: Google LLC
  - IP `1.1.1.1` → ISP: Cloudflare, Inc.

5. Why ISPs Are Important in IP Lookups
- Knowing the ISP can help identify the organization responsible for the IP address.
- It can provide insights into the geographic location and type of network (e.g., residential, business, or mobile).